Detect, respond, coordinate

Class recap

We covered incident management, customer-reported and monitoring-based detection, and why monitoring and alerting are useful. We established on-call rotation and incident commander roles, discussed the commander's responsibilities and actions, and introduced monitoring fundamentals and error rate. GitHub and Billie status pages were shown as examples.

Two routes to discovering an incident

A customer reports a problem. Their experience is evidence of impact. Capture what they tried, what happened and when. A report is worth investigating even when dashboards look healthy.

Monitoring detects a problem. Measurements can reveal failures before a customer reports them. Alerting turns a relevant signal into a notification for someone to act on. A dashboard alone does not ensure anyone notices. [1]

Monitoring can shorten the delay before response, but only if the signal covers the failing behaviour and the notification reaches a responder. It does not guarantee detection of every problem. [1]

A response process to remember

Detect → **assess impact** → **coordinate** → **investigate and mitigate** → **verify recovery** → **learn.** Communicate and keep a short record throughout. Steps can overlap: do not delay safe mitigation while waiting for a complete causal explanation. [2]

Who does what?

On-call responder: the scheduled person who receives and acknowledges the signal, investigates, and calls for help when needed.
Incident commander: coordinates priorities, responsibilities and communication so responders can focus on the technical work. The commander need not personally perform every fix. [2]

Handover reminder: identify unresolved problems, the next action and the person accepting responsibility. Use the rota and communication arrangements agreed in class. This refresher does not change them.

Coordinate and communicate

Incident commander: turning responsibility into action

The class covered what the commander is responsible for and how to act. The following checklist summarises the practice; it is not a claim that we ran a live incident. [2]

Establish command. Explicitly accept the role, identify the communication channel and start a shared incident record. Assess user impact and severity using the agreed criteria; revise the assessment when evidence changes.

Organise the response. Set the immediate priority, assign named owners to investigation, mitigation and communication, and call for additional help when needed. Ask for evidence and next steps; keep technical responders focused rather than attempting every fix yourself.

Maintain a shared picture. Record what is known, what is uncertain, decisions and their reasons. Set the next update time. Coordinate changes so responders do not unknowingly interfere with each other.

Verify and hand over. Ask for evidence that affected behaviour works again. Record remaining risks and follow-up owners. A command handover needs explicit acceptance by the next commander; a proposed handover is not a completed one.

Status pages: the examples from class

We looked at GitHub Status and Billie Status. Both organise service health by component and provide incident history. Use them to examine how an organisation communicates service impact over time. [3, 4]

A status page gives users a shared place to check known impact, current response and recovery updates. An internal dashboard helps responders investigate; a public update should explain the user impact in plain language without exposing sensitive details. A status page should remain reachable when the affected application fails.

Illustrative update: "Some customers cannot complete bookings. We are investigating. Next update at 14:30." State only verified impact, avoid an unsupported recovery promise, and update at the promised time even if investigation continues.

Self-check

Two responders propose different changes while customers ask for updates. What should the commander do? What belongs on the status page, and what belongs in the internal incident record?

Check your reasoning: clarify priorities and owners, coordinate changes and maintain an update cadence. Public communication focuses on impact and service state; the internal record holds detailed evidence, hypotheses and decisions.

Making error rate meaningful

The formula from class

Error rate = number of errors / number of valid events
Multiply the result by 100 to express it as a percentage.

Here, **valid events** means all events eligible for the measurement, including eligible events that failed. It does not mean only successful events. Count errors from that same population and time window. Define which operation is measured and what counts as an error before calculating.

Worked example - illustrative, not a class measurement

Over five minutes, 200 eligible booking attempts occur. Of those, 10 fail and 190 succeed.
10 / 200 = 0.05 = 5%
Dividing by 190 would use successes as the denominator and answer a different question.

Interpret the number

An error percentage needs context: operation, time window, event count and error definition. One failure out of two attempts is 50%; 500 failures out of 1,000 attempts is also 50%, with very different amounts of affected activity. These are examples, not measured course outcomes.

With zero eligible events, the fraction is undefined. Show no traffic or no data as appropriate, rather than claiming 0% errors. Missing telemetry is not proof of a healthy service.

Self-check - no submission required

1. A customer cannot book, but the dashboard is green. What would you investigate?
2. What distinguishes monitoring from alerting?
3. Why separate incident coordination from technical investigation?
4. Of 400 eligible attempts, 12 fail. What is the error percentage?
5. What must you define before comparing two error rates?

Check your reasoning: 1. Examine the reported journey, timing and monitoring coverage. 2. Monitoring observes; alerting notifies. 3. Coordination keeps priorities and communication clear while responders investigate. 4. 3%. 5. The population, error definition, time window and data completeness.

Public references

[1] Google SRE: Monitoring Distributed Systems
[2] Google SRE Workbook: Incident Response
[3] GitHub Status
[4] Billie Status